

Syslab深度学习工具箱

任品

苏州同元软控信息技术有限公司

2025年5月27日



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

课程须知

本课程课程目标:

本课程面向深度学习工具箱新用户，讲述深度学习的基本概念，并且在 Syslab 中演示如何使用工具箱函数求解具体问题，帮助用户对深度学习建立基本了解，并且学会在 Syslab 中使用深度学习工具箱进行深度学习模型的创建、训练、预测。

本课程基于云化版本: Syslab 2024b 构建

本课程需要在首选项加载:

TyDeepLearning

学习本课程之前需要学习:

Syslab基本功能

Julia语法

课程示例的完整代码见附件-深度学习课程示例

CONTENTS

目录

Part 1

背景知识

Part 2

分类与回归

Part 3

聚类

Part 4

图像深度学习

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

Part1

背景知识

1. 人工智能与机器学习
2. 深度学习简介
3. 核心概念

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

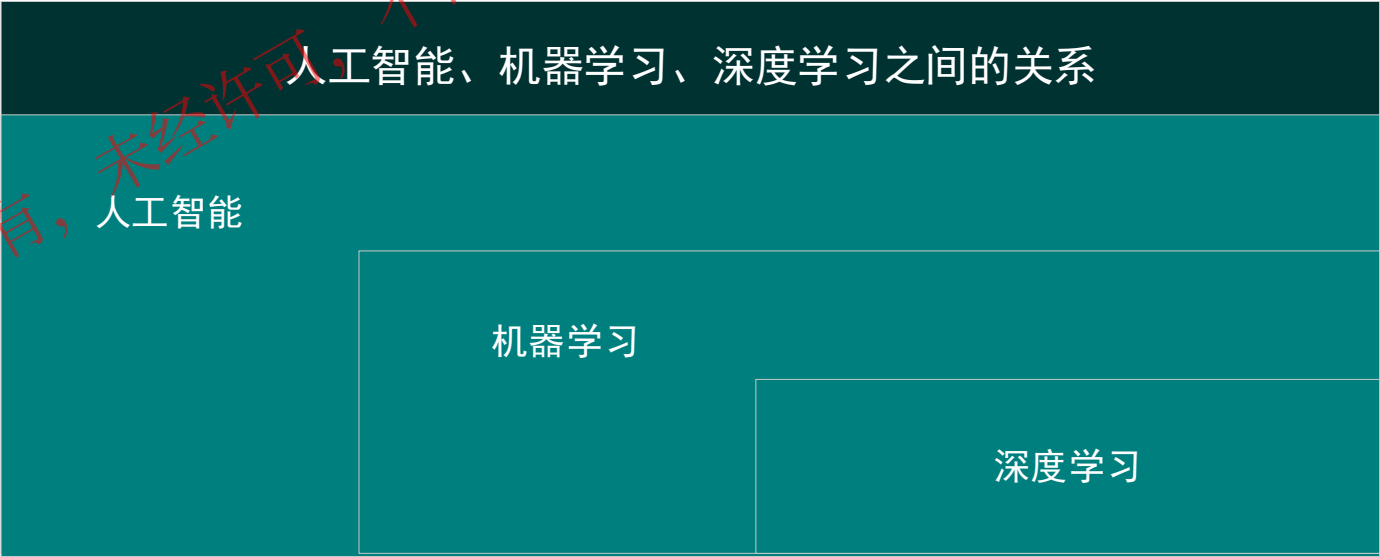
1.人工智能与机器学习

01 人工智能

人工智能 (Artificial Intelligence, AI) 是计算机科学的一个分支，旨在使机器能够执行通常需要人类智能才能完成的任务。

02 机器学习

机器学习是一种通过数据自动学习并改进的技术。它使计算机能够在不进行明确编程的情况下，从数据中提取模式、做出预测或决策。这种能力使得机器学习成为处理复杂、大规模数据集的有力工具。



机器学习是人工智能的一个分支，深度学习是机器学习的一个分支

2.深度学习简介

01 特征提取方式

传统机器学习：依赖人工设计特征，需要领域专家的知识。

深度学习：自动从数据中学习特征，不需要手工干预。

02 模型复杂性

传统机器学习：使用简单模型处理数据。

深度学习：采用深层神经网络处理复杂、高维数据。

03 性能随数据量变化

传统机器学习：数据量有限时表现较好，但随着数据量增加，性能提升有限。

深度学习：数据量越大，性能越强，能够充分发挥其潜力。

04 计算资源需求

传统机器学习：通常对计算资源要求较低。

深度学习：需要强大的计算能力来处理海量数据和复杂模型。

05 应用领域对比

传统机器学习：更适合小样本问题和明确的规则性任务。

深度学习：在复杂模式识别任务中表现卓越。

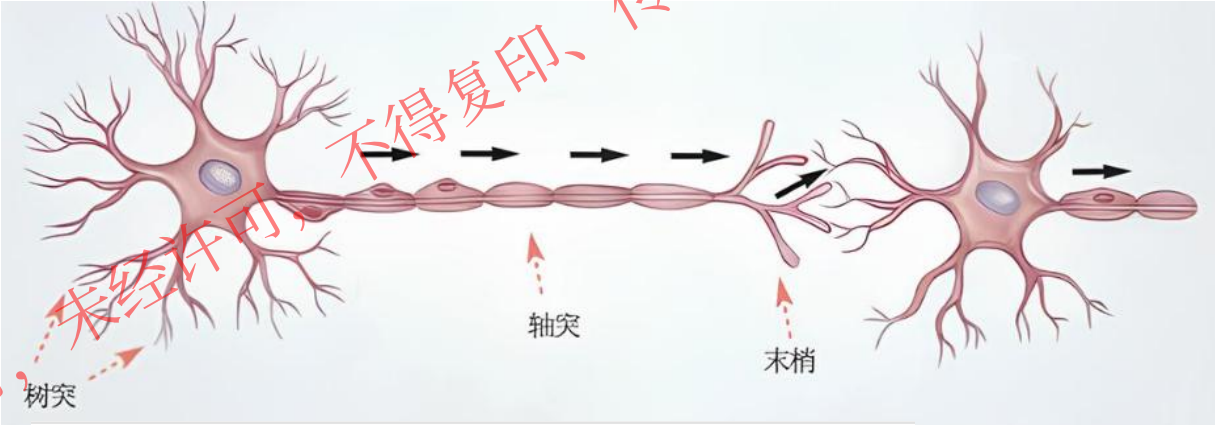
2.深度学习简介

01 深度学习

深度学习是一种机器学习的分支，通过模拟人类大脑的工作方式来处理复杂的数据模式，是利用神经网络进行数据特征学习和决策的技术，适用于解决复杂模式识别问题。

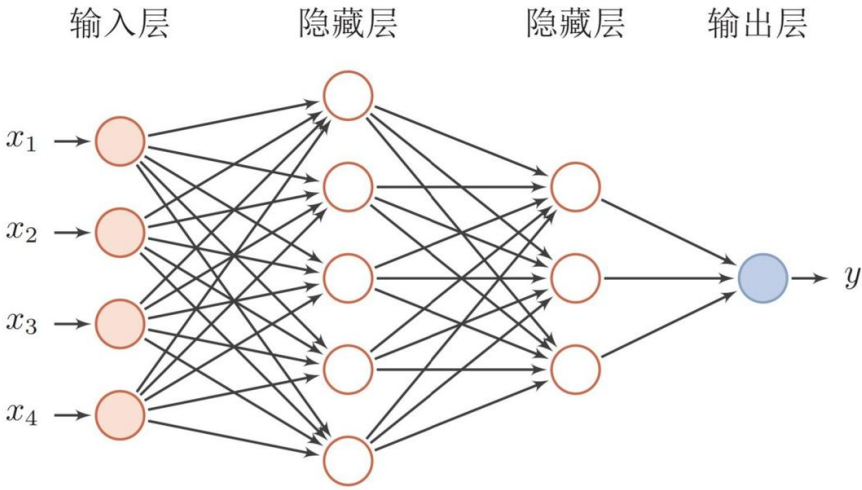
02 神经网络

类比人类大脑：大脑通过神经元之间的连接来处理
和存储信息；同样，神经网络由模拟神经元的“节
点”组成，并通过“权重”进行连接。



03 深度学习与神经网络的关系

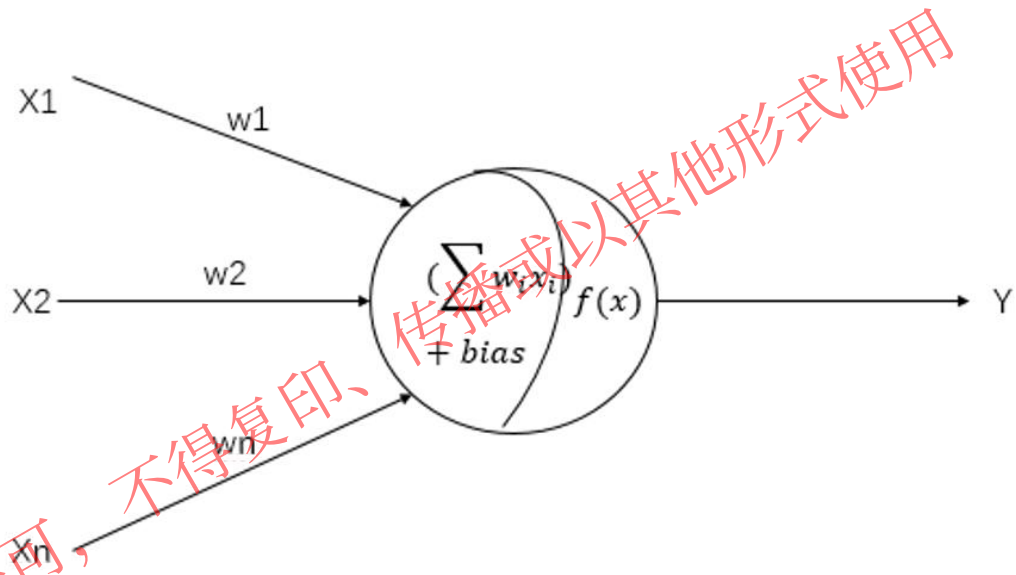
深度学习可以被看作是“深层神经网络”的简称。传统神经网络往往只有1-2层隐藏层，而深度学习通过增加隐藏层的数量，能够学习更复杂、更抽象的特征。通过引入“深度”，神经网络可以模拟更接近人类思维的复杂逻辑。



3.核心概念

01 神经元

神经元是神经网络的基本组成结构。在神经网络中，一个神经元接收信号作为神经元的输入，经过处理，将结果输出，输出的结果将会作为下一个神经元的输入，或者作为最终的输出。

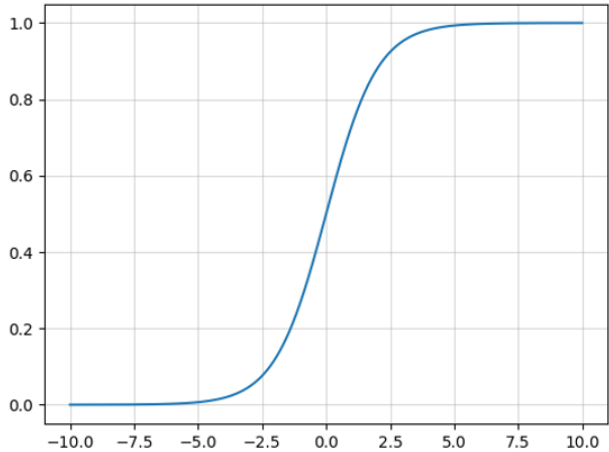


02 权重

当信息作为输入传入到神经元的时候，神经元会分配给每一个信息一个相关权重，然后将输入的信息乘以相应的权重，就是该信息的输入。

03 激活函数

为了引入非线性，神经网络中的每个神经元会通过激活函数转换输出。



04 通用近似定理

对于具有线性输出层和至少一个使用“挤压”性质的激活函数的隐藏层组成的全连接神经网络，只要其隐藏层神经元的数量足够，它可以以任意的精度来近似任何从一个定义在实数空间中的有界闭集函数。

3.核心概念-网络层

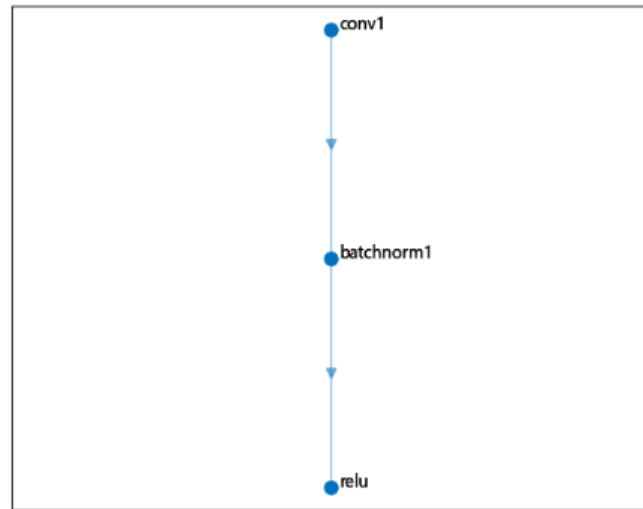
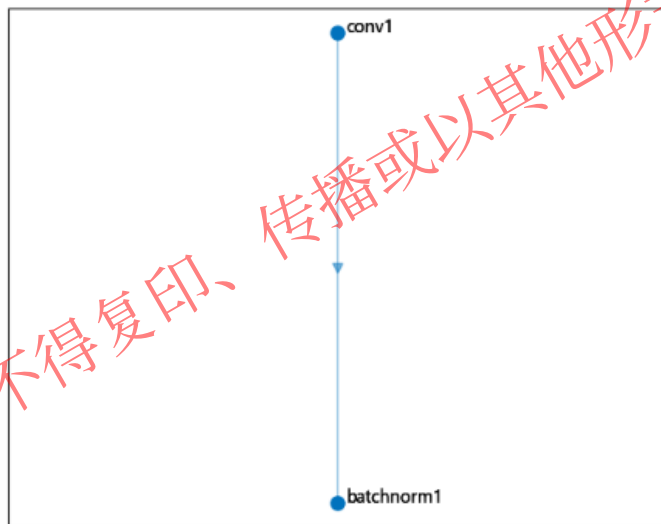
网络层通常由多个神经元组成，通过权重和激活函数共同实现数据的非线性变换。网络层的组合决定了整个神经网络的结构和性能。

创建一个网络图层

```
using TyDeepLearning
layers = [
    ("conv1", convolution2dLayer(1, 16, 3)),
    ("batchnorm1",
    batchNormalization2dLayer(16))]
lgraph = layerGraph(layers)
lgplot(lgraph)
```

添加网络层

```
layer_add = [("relu", reluLayer())]
lgraph = addLayers(lgraph, layer_add)
lgplot(lgraph)
```



Part2

分类与回归

1. 基本概念
2. 分类案例
3. 回归案例

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1.基本概念

01 分类任务

分类是预测离散类别的任务，如判断邮件是否为垃圾邮件，或识别图片中的动物种类。

02 回归任务

回归是预测连续值的任务，如预测房价、温度或股票价格。

03 损失函数

当建立一个神经网络的时候，神经网络会试图将输出预测尽可能地接近实际值。使用损失函数 (Cost function) 来衡量网络的准确性。

04 梯度下降

梯度下降是最小化损失的优化算法，找到最小化的损失函数和模型参数值。

2.分类案例

1.导入鸢尾花数据。

```
using TyDeepLearning
using CSV
using DataFrames
set_backend(:mindspore)
file = dataset_dir("iris")
input_path = file*"/irisInputs.csv"
target_path = file*"/irisTargets.csv"
input = CSV.read(input_path, DataFrame; header = false)
target = CSV.read(target_path, DataFrame; header = false)
input = Array(input)
target = Array(target)
```

3.网络训练

```
net_trained = trainNetwork(input, target, net, options)
```

2.定义一个神经网络，并指定网络训练的选项。

```
net = SequentialCell([
    fullyConnectedLayer(4, 20),
    sigmoidLayer(),
    fullyConnectedLayer(20, 3),
    softmaxLayer()
])
options = trainingOptions("CrossEntropyLoss",
    "Adam", "Accuracy", 30, 1000, 0.01; Plots=true)
```



山鸢尾(*Iris setosa*)



变色鸢尾(*Iris versicolor*)



维吉尼亚鸢尾(*iris virginica*)

分类特征：花萼（sepal）和花瓣（petal）的宽度和长度

2.分类案例

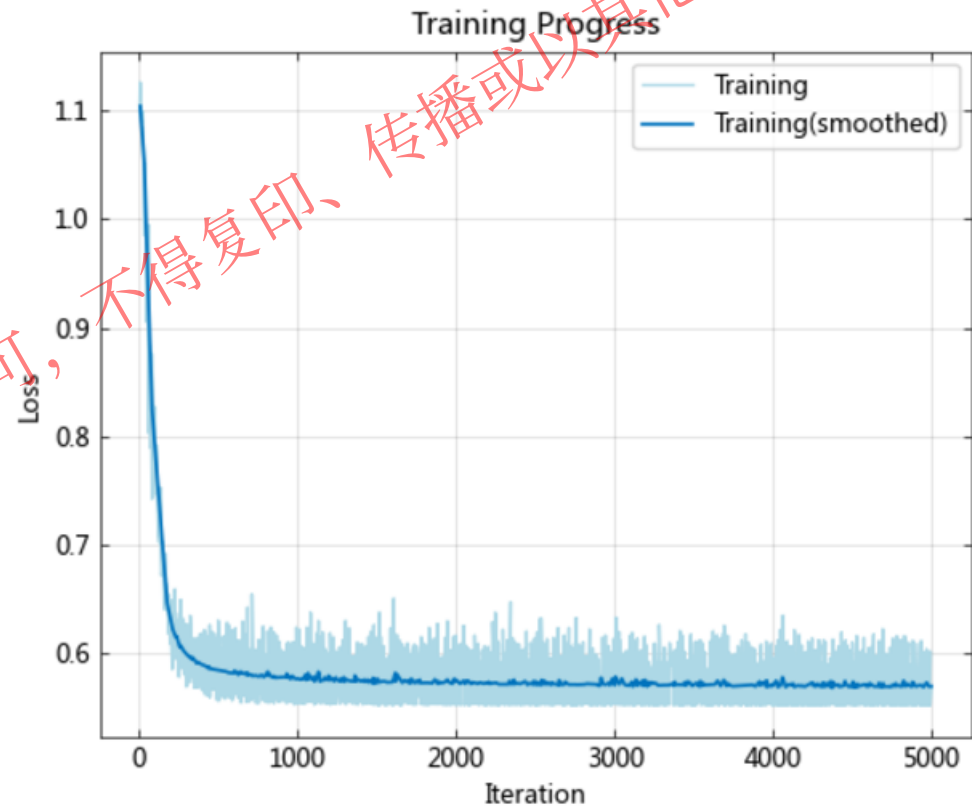
4.使用训练好的网络，对测试集进行分类预测并计算混淆矩阵。

```
prob = TyDeepLearning.classify(net_trained, input)
classes = [i - 1 for i in range(1, 3)]
YPred = probability2classes(prob, classes)
cm = confusionchart(target[:, 1], YPred)
```

3x3 Matrix{Float64}:

50.0	0.0	0.0
0.0	47.0	0.0
0.0	3.0	50.0

神经网络每次训练具有随机性，输出结果可能存在差异。



3.回归案例

加载数据

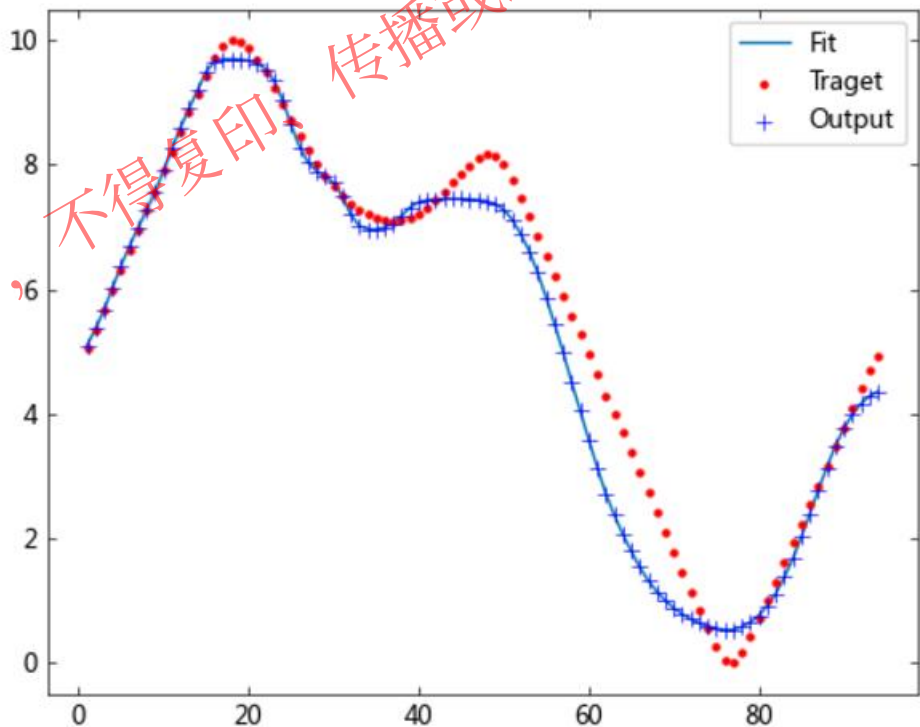
```
using TyDeepLearning
file = dataset_dir("simplefit")
x, t = simplefit_dataset(file)
```

训练一个前馈神经网络

```
hiddenSizes = 10
net = feedforwardnet(hiddenSizes)
net_trained = train(net, x, t; epochs = 2000, lr = 0.05)
```

使用已训练的网络进行预测，绘制输出和目标值拟合图。

```
TyDeepLearning.plotfit(net_trained, x, t)
```



神经网络每次训练具有随机性，输出结果可能存在差异。

Part3

聚类

1. 基本概念
2. 算法介绍
3. 聚类案例

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1.基本概念

01 定义

聚类是一种无监督学习方法，通过将相似的数据点分组形成簇，使得簇内样本相似性最大化，簇间样本差异性最大化。

02 目的与特点

将样本点归类到不同簇，发现数据的内在结构，无需预先提供标签。

03 典型应用

客户分群：根据购买行为将客户分组。

图像分割：将像素分为不同区域。

基因表达数据分析：对基因样本进行分组。

2.算法介绍

01 K-means聚类

基于质心的聚类，算法过程：

- 1) 随机初始化 k 个聚类中心。
- 2) 分配数据点到最近的聚类中心。
- 3) 重新计算每个聚类的质心。
- 4) 重复步骤2和3，直到质心不再变化或达到迭代次数。

02 基于密度的聚类 (DBSCAN)

通过检测点的密度分布确定簇，不需要事先指定簇的数量。
适合不规则形状的簇，同时可以发现离群点

03 层次聚类

通过构建层次树状结构实现聚类。

分裂式聚类： 从一个整体簇开始，逐步分裂成小簇。

凝聚式聚类： 每个样本点开始为一个簇，逐步合并。

04 自组织映射 (SOM)

基于神经网络的聚类算法，模拟大脑皮层神经元的自适应特性。

- 1) 初始化：随机生成二维网格中每个神经元的权重向量。
- 2) 对于每个输入数据点：计算每个神经元与数据点的距离，选出距离最小的神经元（最佳匹配单元，BMU）。更新BMU及其邻域内神经元的权重，使其更接近输入数据。
- 3) 随着迭代进行，邻域半径和学习率逐步减小。

3. 聚类案例

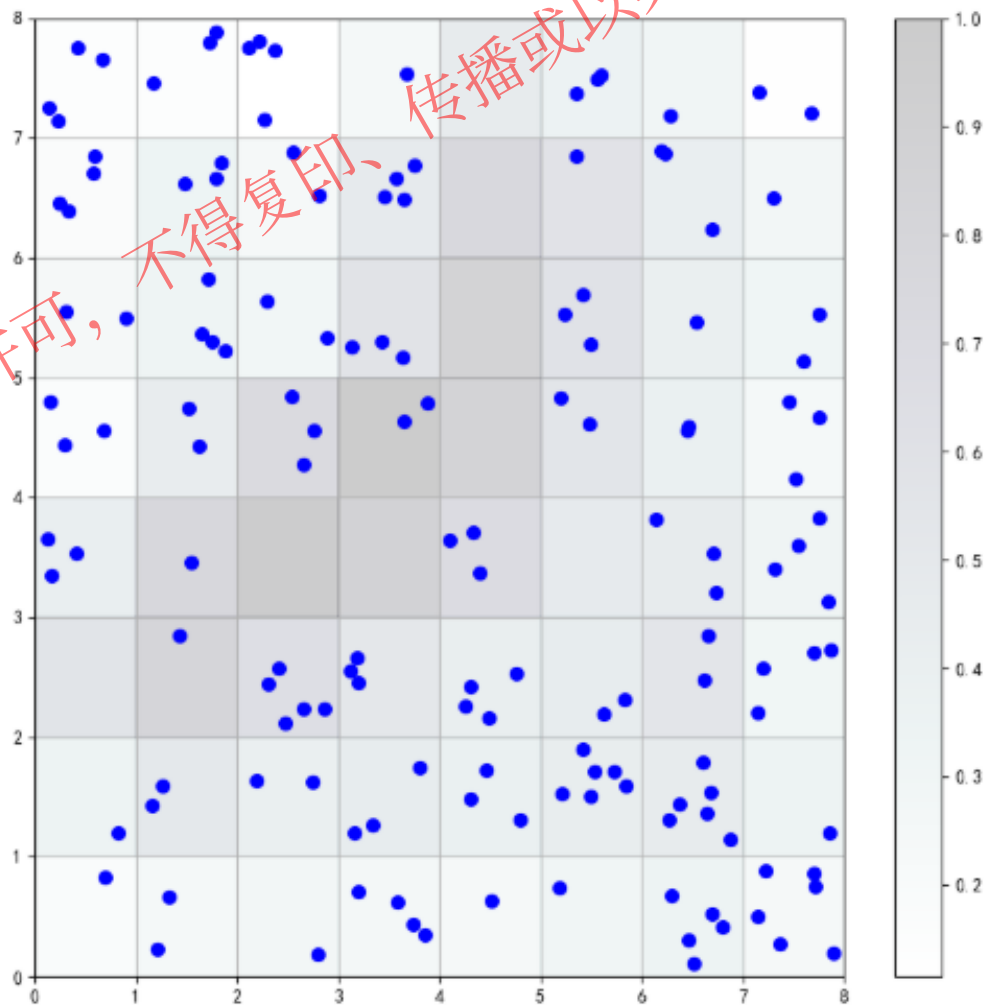
数据：使用鸢尾花数据集。

目标：四个花属性将作为 SOM 的输入，SOM 将它们映射到二维神经元层。

```
#加载数据
using TyDeepLearning
file = dataset_dir("iris")
x, t = iris_dataset(file)
```

```
#创建自组织映射网络并训练
net = selforgmap(8, 8, 4);
net.train(x, 100, verbose=true)
```

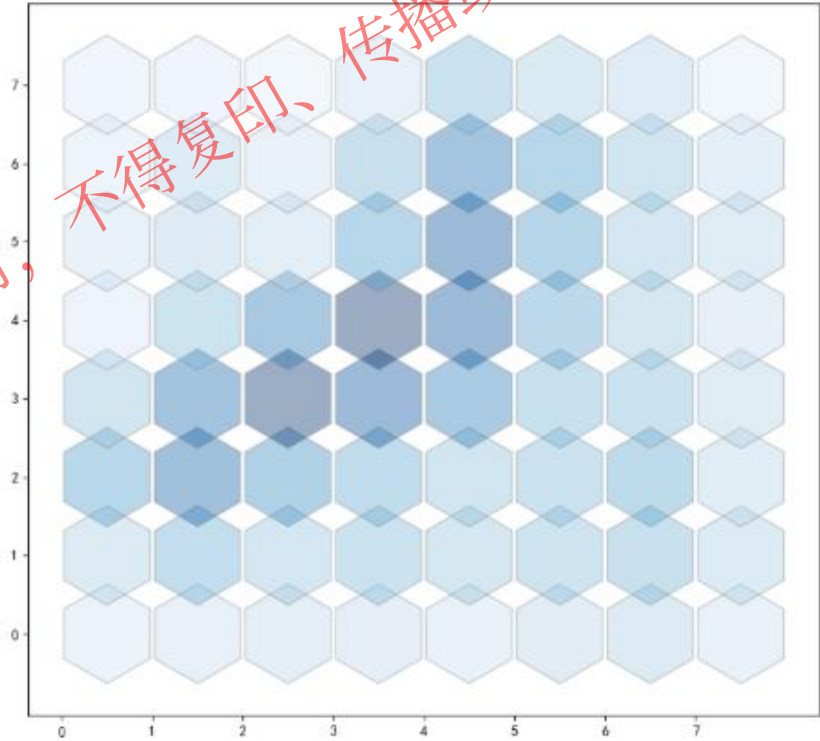
```
#绘制自组织映射样本命中
plotsomhits(net, x)
```



3. 聚类案例

```
#绘制自组织映射相邻距离  
plotsomnd(net,x)
```

浅色连接表示输入空间的高度连接区域。
深色连接表示的类代表相距很远且相互之间很少或没有鸢尾花的特征空间区域。



Part4

图像深度学习

1. 基本概念
2. 角度预测案例
3. 图像分类案例

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

1.基本概念

01 定义

图像深度学习是利用深度学习技术对图像数据进行处理和分析，常用于分类、检测、分割等任务。

02 特点

维度高： 图像通常是二维甚至三维数据，包含大量像素点，每个像素可能有多个通道（如RGB）。相比于传统结构化数据（如表格数据），维度更高、特征更复杂。

局部性特征： 图像具有空间信息，局部像素之间的关联性是分析的关键。这需要专门的模型（如卷积神经网络）来提取局部和全局特征。

03 卷积神经网络（CNN）

是一种专门用于处理图像数据的深度学习模型，因其对空间结构的处理能力和高效性而被广泛应用。

局部感受野： CNN 的每个神经元只处理输入数据的局部区域而不是全局数据。这种设计减少了计算量并保留了空间信息。

权值共享： 同一个卷积核在不同位置重复使用，能够识别图片中相同的特征，同时显著减少参数数量。

工作原理：

输入一张图片，经过卷积层提取低级特征。

多次卷积后逐渐提取更高级特征。

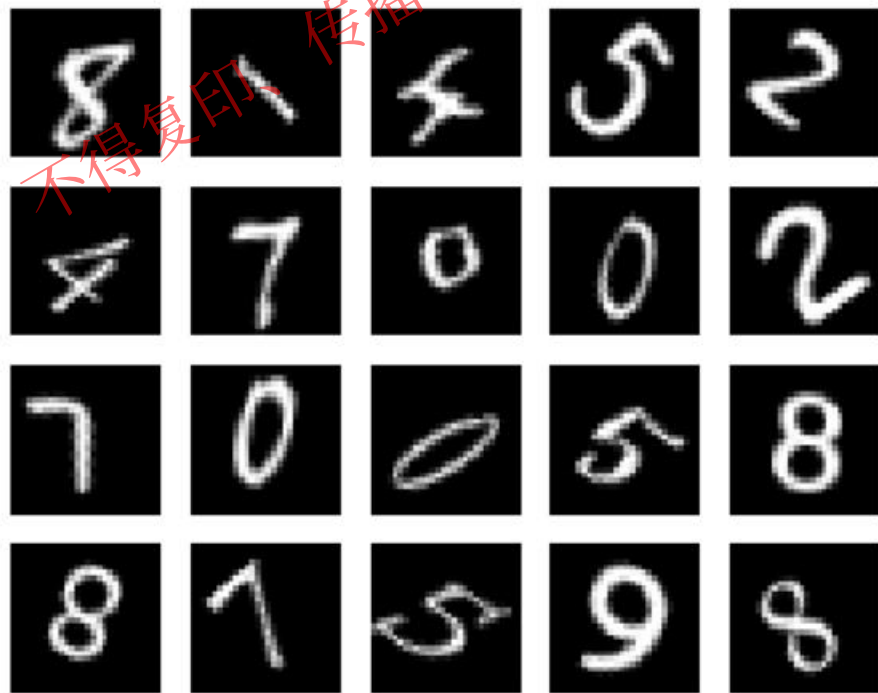
最后通过全连接层输出结果。

2.角度预测案例

加载手写数字数据集

```
using TyDeepLearning
using TyImages
using TyPlot
using TyMath
file = dataset_dir("digit")
XTrain, YTrain = DigitTrain4DArrayData(file)
```

```
# 随机选择 20 张图像进行绘制
index = randperm(5000)[1:20]
figure(1)
for i in eachindex(range(1, 20))
    subplot(4, 5, i)
    imshow(XTrain[index[i], 1, :, :])
end
```

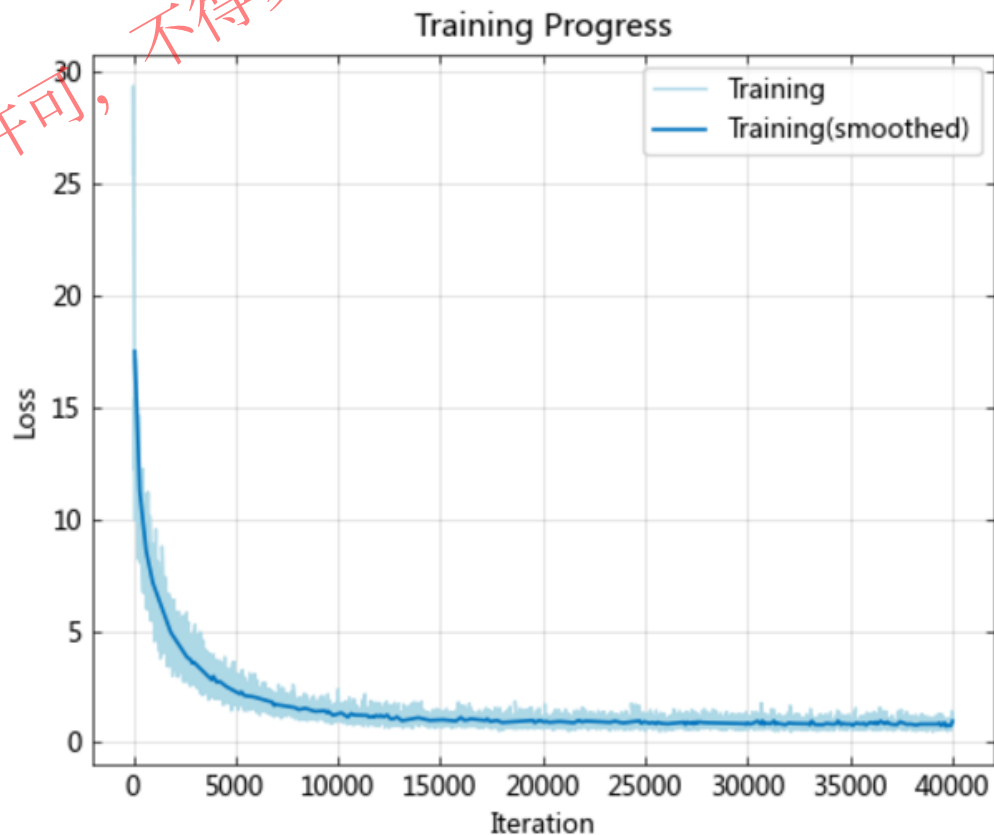


2.角度预测案例

```
# 指定卷积神经网络体系结构。
layers = SequentialCell([
    convolution2dLayer(1, 25, 12),
    reluLayer(),
    flattenLayer(),
    fullyConnectedLayer(25 * 28 * 28, 1),
])
```

```
# 将数据集划分为训练集和测试集
using Random
p = randperm(5000)
index1 = p[1:1000]
index2 = p[1001:end]
X_train = XTrain[index2, :, :]
Y_train = YTrain[index2, :]
X_test = XTrain[index1, :, :]
Y_test = YTrain[index1, :]
```

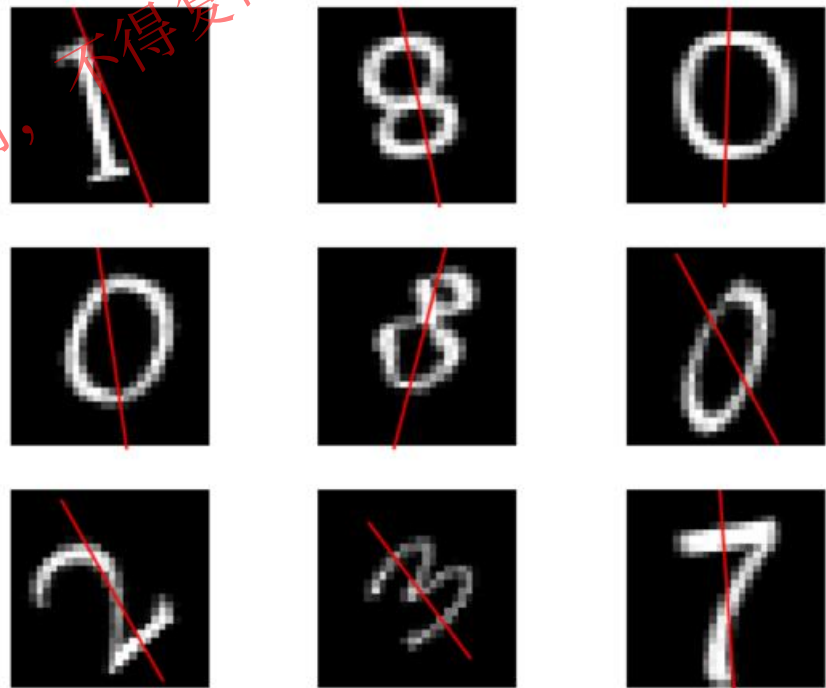
```
#指定网络训练选项，训练网络
options = trainingOptions("RMSELoss", "Adam", "MSE",
50, 500, 0.0001; Plots=true)
net = trainNetwork(X_train,Y_train,layers,options);
```



2.角度预测案例

```
YPred = TyDeepLearning.predict(net, X_test)
rmse = sqrt(mse(Y_test, YPred))
index = randperm(1000)[1:9]
figure(3)
for i in range(1, 9)
    subplot(3, 3, i)
    hold("on")
    imshow(X_test[index[i], 1, :, :])
    x = [7:21...]
    plot(x, tan((90 + YPred[index[i]]) / 180 *
pi) * (x .- 14) .+ 14, "r")
    ax = gca()
    ax.set_ylim(28,0)
    ax.set_xlim(0, 28)
    hold("off")
end
```

```
julia> rmse
7.60753f0
```

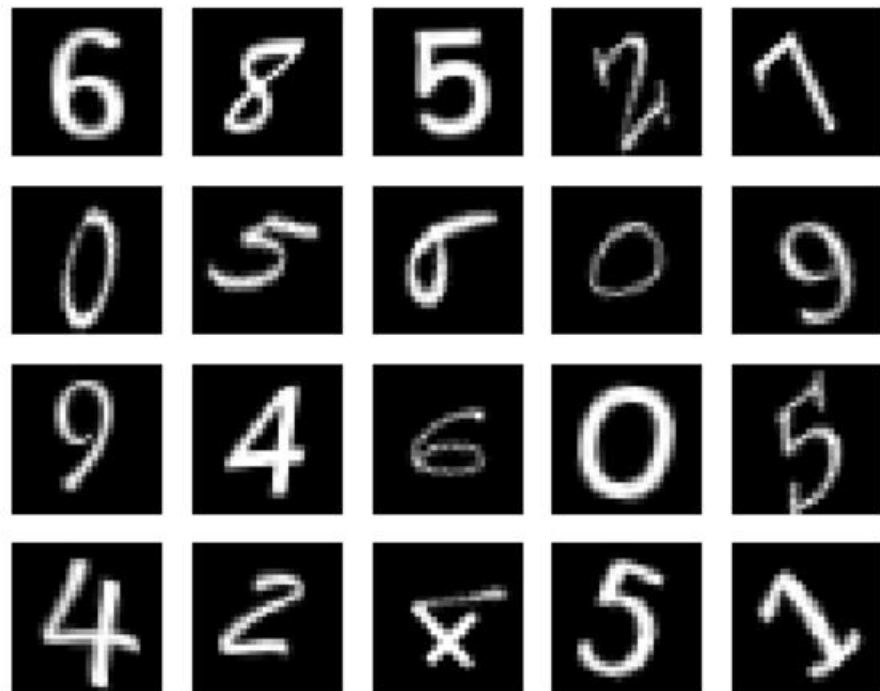


3.图像分类案例

```
# 加载手写数字数据集
using TyDeepLearning
file = dataset_dir("digit")
XTrain, YTrain = DigitDatasetTrainData(file)
```

```
# 随机选择 20 张图像进行绘制
using TyPlot
using TyImages
N, Channel, Hight, Width = size(X_train)
p2 = randperm(N)
index = p2[1:20]
figure(1)
for i in eachindex(range(1, 20))
    subplot(4, 5, i)
    imshow(X_train[index[i], 1, :, :])
end
```

```
# 将数据集划分为训练集和测试集
using Random
p = randperm(5000)
index1 = p[1:1000]
index2 = p[1001:end]
X_train = XTrain[index2, :, :, :]
Y_train = YTrain[index2, :]
X_test = XTrain[index1, :, :, :]
Y_test = YTrain[index1, :]
```



3.图像分类案例

```
# 图像增强处理, 将图片在[-20, 20]度范围内随机旋转
imageAugmenter =
imageDataAugmenter(RandomRotation = 20)
X_train2 = permutedims(X_train, (1, 3, 4, 2))
imagesize = (28, 28)
augimds = augmentedImageDatastore(imagesize,
X_train2, Y_train, imageAugmenter)
```

```
# 绘制图像增强处理后的图像。
augimds = permutedims(augimds, (1, 4, 2, 3))
figure(2)
for i in eachindex(range(1, 20))
    subplot(4, 5, i)
    imshow(augimds[index[i], 1, :, :])
end
```

本页代码为伪代码, 运行可使用附件代码案例



3.图像分类案例

指定网络训练选项。使用 CrossEntropyLoss 作为损失函数，Adam 为优化算法，评价指标为准确度，学习率为 0.01，epoch 为 100，batchsize 为 64。

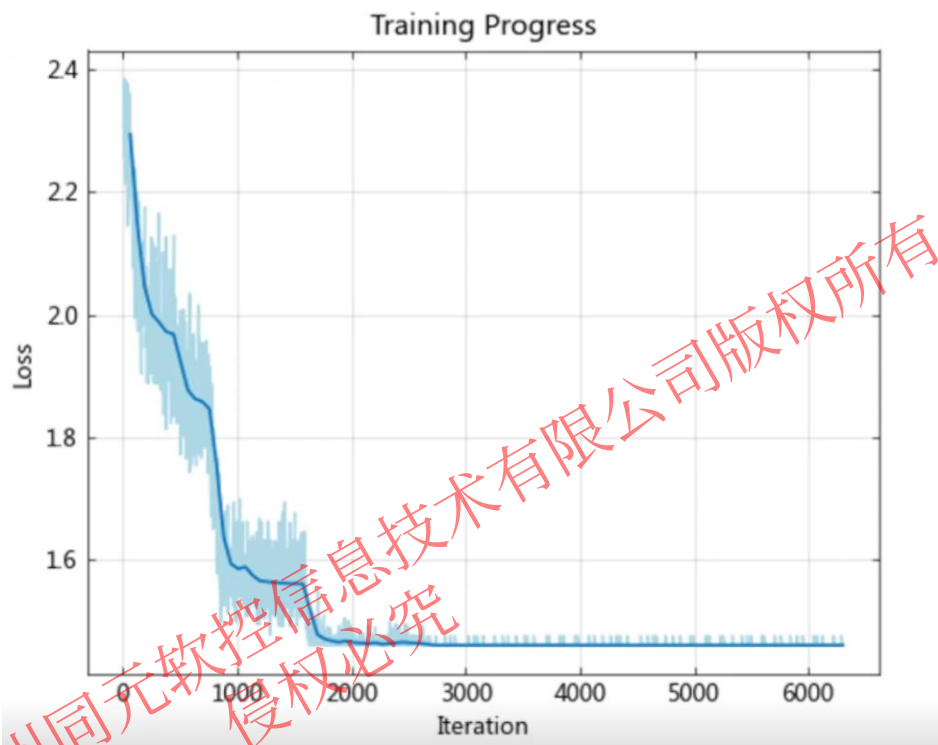
```
options = trainingOptions("CrossEntropyLoss", "Adam", "Accuracy", 64, 100, 0.01; Shuffle
=false , Plots = true)
```

构建网络结构

```
layers = SequentialCell([
    convolution2dLayer(Channel, 8, 3), batchNormalization2dLayer(8),
    reluLayer(),
    maxPooling2dLayer(2; Stride = 2),
    convolution2dLayer(8, 16, 3), batchNormalization2dLayer(16),
    reluLayer(),
    maxPooling2dLayer(2; Stride = 2),
    convolution2dLayer(16, 32, 3), batchNormalization2dLayer(32),
    reluLayer(),
    flattenLayer(),
    fullyConnectedLayer(32 * 7 * 7, 10),
    softmaxLayer()])
```

3.图像分类案例

```
# 训练网络  
net = trainNetwork(augimds, Y_train, layers, options)
```



3.图像分类案例

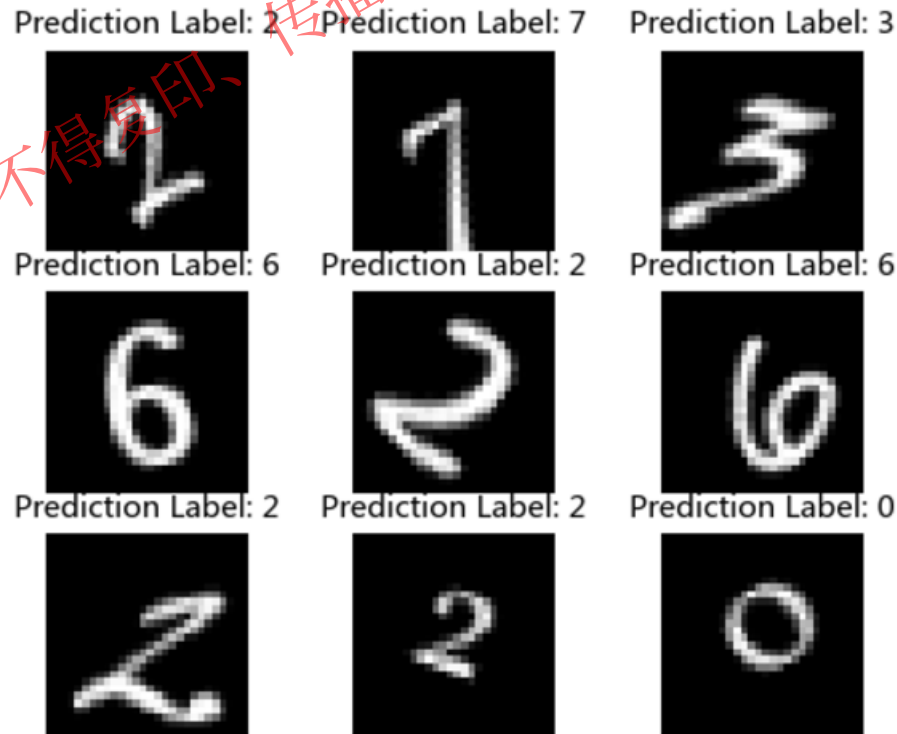
查看训练效果

```
YPred = TyDeepLearning.predict(net, X_test )
Y_test = reshape(Y_test, (1000))
acc = accuracy(YPred, Y_test)
```

在测试集随机取9张图片进行展示，并使用网络对图片类别进行预测。

```
classes = [i - 1 for i in range(1, 10)]
YPred1 = probability2classes(YPred, classes)
figure(4)
p2 = randperm(1000)
index = p2[1:9]
for i in eachindex(range(1, 9))
    TyPlot.subplot(3, 3, i)
    TyImages.imshow(X_test[index[i], 1, :, :])
    title1 = "Prediction Label"
    title2 = string(YPred1[index[i]])
    title(string(title1, ": ", title2))
end
```

```
julia> acc
0.999
```



神经网络每次训练具有随机性，输出结果可能存在差异。

感谢您的聆听!

Thanks



未经授权，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究